

TODO – Ego-Only VideoMAE Variant (Keep Existing Exo Baseline)

This TODO is for an AI coding agent working in the `TrackABC` repo.

High-level goal

- Keep the current **exo-transfer 2D backbone + fusion** pipeline as the **baseline**.
 - Add a new **Ego-only VideoMAE video backbone** trained on **in-domain egocentric clips** (Ego4D / STA).
 - Plug this VideoMAE backbone into Track B (and optionally Track C) and **compare metrics** on STA v2 val.
-

0. Repo reconnaissance

- ☐ Locate the core track code:
 - ☐ `trackA/` (Track A detector / head proposals, YOLO-based).
 - ☐ `trackB/` (Track B dataset, tokenizer, fusion, head, training, evaluation).
 - ☐ `trackC/` (Track C pruning / RGTP code).
- ☐ Confirm these key files exist and open them:
 - ☐ `trackB_dataset.py`
 - ☐ `trackB_tokenizer.py`
 - ☐ `trackB_fusion.py`
 - ☐ `trackB_head.py`
 - ☐ `trackB_train.py` (or equivalent training script)
 - ☐ `trackB_eval.py`
 - ☐ `trackC_pruning.py`
- ☐ Verify that currently:
 - ☐ Frames are encoded with a **2D image backbone** (CLIP/ViT-style).
 - ☐ Temporal information is handled by **stacking per-frame tokens** and applying **fusion**.

Do **not** delete or break the existing CLIP/2D backbone path. We will add a *second* path.

1. Make the current exo-transfer pipeline an explicit “baseline”

- ☐ Introduce a simple configuration flag to distinguish backbones, e.g. in `TokenizerConfig` or a central config file:
 - ☐ Add a field like: `video_backbone: str = "clip_image"` (values: `"clip_image"`, `"videomae_ego"`).
 - ☐ Ensure the **existing behaviour** remains the default:
 - ☐ When `video_backbone == "clip_image"`, use the current per-frame image encoder + fusion exactly as now.
 - ☐ Add a comment in the config explaining:
 - `"clip_image"` = current exocentric / web-pretrained 2D backbone (baseline).
 - `"videomae_ego"` = new in-domain VideoMAE backbone (to be implemented).
-

2. Build an Ego-only VideoMAE pretraining pipeline

Create a **separate pretraining script** that does *self-supervised* VideoMAE on egocentric clips.

2.1. Dataset of in-domain clips (around 95Gb clips --- "H:\My Drive\ego4d_data\v2\clips_540\clips")

- ☐ Create a new file, e.g. `videomae_pretrain_dataset.py`.
- ☐ Implement a `Dataset` that:
 - ☐ Consumes **STA v2 train** data (and optionally other ego datasets, e.g. EPIC).
 - ☐ For each STA clip UID:
 - ☐ Samples a short window of **T frames** (e.g. 16 or 32) from the corresponding video or extracted frames.
 - ☐ Returns a tensor of shape `(T, C, H, W)` or `(C, T, H, W)` depending on the VideoMAE implementation.
 - ☐ Applies basic augmentations (resize/crop/flip, etc. consistent with VideoMAE examples).

2.2. VideoMAE model

- ☐ Create `videomae_model.py` (or similar) that:
 - ☐ Either:
 - ☐ Uses an existing VideoMAE implementation from `timm` / public code (if accessible),
 - ☐ Or implements a minimal VideoMAE encoder + decoder based on standard Masked Autoencoder logic.
 - ☐ Clearly separate:
 - ☐ **Encoder**: returns spatiotemporal tokens (this is what Track B will reuse).
 - ☐ **Decoder**: only needed for reconstruction during pretraining.

2.3. Pretraining script

- ☐ Add `videomae_pretrain.py`:
 - ☐ Build the Ego dataset from STA train split.
 - ☐ Instantiate VideoMAE encoder/decoder.
 - ☐ Implement masked autoencoding:
 - ☐ Randomly mask a large fraction of spatiotemporal patches.
 - ☐ Reconstruct the original frames from the visible tokens.
 - ☐ Train for a reasonable number of epochs on GPU.
 - ☐ Save a checkpoint, e.g. `local_extraction/runs/VideoMAE/videomae_ego_encoder.pt` containing:
 - ☐ Encoder weights.
 - ☐ Config (patch size, tubelet size, input resolution, etc.).

This step is **self-supervised**: use only raw frames, no labels.

3. Integrate VideoMAE encoder into Track B

Goal: add an option so Track B can use **VideoMAE** instead of per-frame CLIP for the **video branch**, while keeping everything else (fusion, head) as similar as possible.

3.1. Tokenizer extension

- ☐ Open `trackB_tokenizer.py`.
- ☐ Add support for a new mode in `TokenizerConfig`, e.g.:
 - ☐ `video_backbone: Literal["clip_image", "videomae_ego"] = "clip_image"`.
- ☐ Implement a new function, e.g. `encode_video_clip_videomae(...)`, that:
 - ☐ Loads the pretrained VideoMAE encoder weights from `videomae_ego_encoder.pt`.
 - ☐ Accepts a **T-frame clip** for each STA candidate (or for the whole frame context).
 - ☐ Returns a set of **spatiotemporal tokens**:
 - ☐ Shape something like `(T * N_patches, D)` or `(N_tokens, D)` compatible with `TrackBFusion`.
- ☐ In the main tokenizer logic:
 - ☐ If `video_backbone == "clip_image"` → keep the existing behaviour.
 - ☐ If `video_backbone == "videomae_ego"` → call the new VideoMAE encoder instead of the image backbone for the video tokens.

3.2. Fusion + head compatibility

- ☐ Check `trackB_fusion.py`:
 - ☐ Confirm it can operate on the tokens from VideoMAE (just another `(N_tokens, D)` sequence).
 - ☐ If needed, adjust to handle slightly different temporal token shapes (e.g. `(T, H*W, D)` vs `(N_tokens, D)`).
- ☐ Check `trackB_head.py`:
 - ☐ Ensure dimensions align (`dim == token_dim`).
 - ☐ No major logic change should be necessary; it just receives fused tokens.

3.3. Configs for the two variants

- ☐ Add two config presets (YAML or Python dataclasses, depending on your style):
 - ☐ **Baseline exo-transfer config**, e.g. `configs/trackB_clip_baseline.yaml`
 - ☐ `video_backbone: "clip_image"`
 - ☐ Same hyperparams as your current best run.
 - ☐ **Ego-VideoMAE config**, e.g. `configs/trackB_videomae_ego.yaml`
 - ☐ `video_backbone: "videomae_ego"`
 - ☐ `token_dim` consistent with VideoMAE encoder output.
 - ☐ Otherwise as close as possible to the baseline (batch size, LR, etc.).

4. Train Track B with the new backbone

- ☐ Modify `trackB_train.py` (or equivalent) so it can:
 - ☐ Read a `--config` or flags to switch between CLIP baseline and VideoMAE variant.
- ☐ Run **two training runs** on STA v2 train:

- ☐ **Run 1 – Baseline (already done, but re-run is ok)**
 - ☐ Config: `video_backbone="clip_image"` .
 - ☐ Save checkpoint as you currently do (e.g. `trackB_best_clip.pt`).
- ☐ **Run 2 – Ego VideoMAE**
 - ☐ Config: `video_backbone="videomae_ego"` .
 - ☐ Load `videomae_ego_encoder.pt` and keep encoder frozen or lightly fine-tuned (your choice, but document it).
 - ☐ Save checkpoint as `trackB_best_videomae_ego.pt` .

5. Evaluate and compare on STA v2 val

- ☐ Use `trackB_eval.py` with each checkpoint:
 - ☐ Baseline:
 - ☐ `checkpoint = trackB_best_clip.pt`
 - ☐ `video_backbone="clip_image"`
 - ☐ Ego-VideoMAE:
 - ☐ `checkpoint = trackB_best_videomae_ego.pt`
 - ☐ `video_backbone="videomae_ego"`
- ☐ For each run, record:
 - ☐ `accuracy`
 - ☐ `mAP`
 - ☐ `ttc_mae_seconds`
 - ☐ `N_mAP` , `Nv_mAP` , `N_delta_mAP` , `All_mAP` (if available)
- ☐ Write a small script, e.g. `summarize_ablation.py` , that:
 - ☐ Reads metrics JSONs from `local_extraction/runs/Track_B/metrics` .
 - ☐ Produces a Markdown table comparing:

Model variant	video backbone	mAP	TTC MAE	N mAP	Nv mAP	N+δ mAP	All mAP
Exo-transfer CLIP (base)	clip_image	...	...	...	...	...	...
Ego-only VideoMAE (ours)	videomae_ego	...	...	...	...	...	...

- ☐ Save this table as `results/ablation_videomae_vs_clip.md` .

6. Optional: hook VideoMAE into Track C pruning

If bandwidth allows, extend Track C to also support VideoMAE tokens.

- ☐ Update `trackC_pruning.py` to respect `video_backbone` from a config:
 - ☐ If `clip_image` → behaviour unchanged.
 - ☐ If `videomae_ego` → load VideoMAE encoder as in Track B and produce video tokens accordingly.
 - ☐ Run Track C with:
 - ☐ Baseline backbone.
 - ☐ Ego-VideoMAE backbone.
 - ☐ Add their results to the same ablation table or a new one.
-

7. Documentation for the thesis

- ☐ Add/extend `TrackABC_readme.md` (or a new `videomae_readme.md`) with:
 - ☐ Short description of the **baseline exo-transfer** setup:
 - ☐ 2D image backbone (CLIP/ViT), per-frame encoding, temporal fusion.
 - ☐ Short description of the **Ego-only VideoMAE** setup:
 - ☐ Self-supervised pretraining on in-domain ego clips.
 - ☐ Same Track B head/fusion; only the video backbone differs.
 - ☐ A section explaining:
 - ☐ The motivation: “in-domain vs exocentric transfer”.
 - ☐ Training details (clip length, masking ratio, pretraining epochs).
 - ☐ Final ablation table from `results/ablation_videomae_vs_clip.md` .
 - ☐ Make sure to explicitly state in the doc:
 - ☐ “Our original CLIP/2D backbone pipeline is kept as a **strong baseline**.”
 - ☐ “We add an Ego-only VideoMAE video branch and show a direct comparison on STA v2 val.”
-